# Algorithmic Robotics
# COMP/ELEC/MECH 450/550
# Project 3: Barking up a Random Tree

Darshon Singh (ds161) and Winson Lin (wl85)

## Problem

We implemented the Random Tree Planner (RTP) in C++ as an OMPL planner and used it to plan collision free motions for the seven joint Panda arm from Project 2. We made two of our own 3D scenes where the straight joint space line from start to goal hits an obstacle and ran RTP on them. Then we benchmarked RTP against OMPL's RRT, RRT-Connect and PRM on the five queries of the supplied clutter scene, with several parameter settings each, and compared planning time, path length, smoothness, minimum clearance and exact success rate.

## Panda arm and its joint space

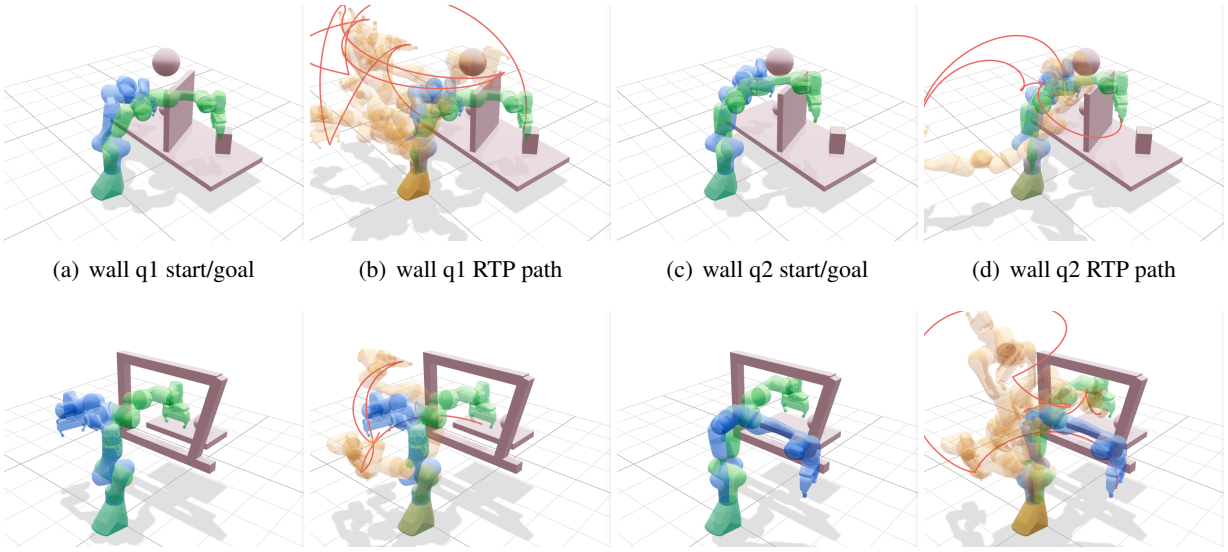
The Panda is a 7 joint revolute arm with a fixed base and gripper, so a configuration is just the seven joint angles $q = (q_1, \ldots, q_7)$ and the configuration space is the box $\mathcal{C} = \prod_{i=1}^{7} [q_i^{min}, q_i^{max}] \subset \mathbb{R}^7$ with these URDF limits (radians):

| joint | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| min | $-2.97$ | $-1.83$ | $-2.97$ | $-3.14$ | $-2.97$ | $-0.09$ | $-2.97$ |
| max | 2.97 | 1.83 | 2.97 | 0.09 | 2.97 | 3.82 | 2.97 |

It is set up as `RealVectorStateSpace(7)` and local motions are straight lines between joint vectors. For collision checking the arm is 59 spheres attached to the links. Forward kinematics puts them in the world frame and they get checked against the scene spheres and oriented boxes (no self collision). The longest diagonal of the joint box is about 13.4 rad, so the fixed edge resolution of 0.005 means edges are checked about every 0.067 rad.

## RTP implementation

RTP only uses OMPL's state space, validity checker and goal, so it knows nothing about the arm. The tree is a `std::vector` of `Motion` structs (state plus parent pointer). Since RTP picks tree vertices uniformly, a random index into the vector is enough and there is no nearest neighbor structure. Each iteration picks a random vertex $q_a$, samples $q_b$ uniformly (or takes the goal with probability equal to the goal bias), and calls `checkMotion(`$q_a$`,` $q_b$`)` on the whole edge. If it is valid, $q_b$ is cloned and added with parent $q_a$. We stop when a new vertex satisfies the goal or time runs out, and get the path by walking parent pointers back to the start. We also track the vertex closest to the goal, and if time runs out we return the path to it as an



(a) wall q1 start/goal (b) wall q1 RTP path (c) wall q2 start/goal (d) wall q2 RTP path

(e) window q1 start/goal (f) window q1 RTP path (g) window q2 start/goal (h) window q2 RTP path

**Figure 1:** Custom scenes in Viser. Blue is the start, green the goal, faded orange the RTP waypoints, red the gripper path. RTP used goal bias 0.05 and a 10 s limit.

approximate solution. The planner owns every state and `Motion` it allocates and `freeMemory()` releases them. Both `clear()` and the destructor call it, which matters because the benchmark reuses one planner instance across trials. `getPlannerData()` adds every tree edge and labels the start and goal vertices.

To check it we ran RTP on all supplied clutter and shelf queries with 5 seeds each and densely rechecked every returned edge. All 50 runs were exact and valid (one shelf path had about 1 mm of penetration between two validator steps, which comes from the fixed checking resolution). With a 2 ms limit RTP correctly returns an approximate solution.

## Custom scenes and RTP paths (Exercise 2)

Wall scene (`wall.yaml`): a table with a thin wall on it, rotated about 25 degrees around $z$, splitting it into a left and a right side, plus a ball hanging just above the wall at 0.78 m so the arm can't just go straight over. A small ball and a rotated block add clutter. Both queries start with the hand pointing down on the left side and end on the right side: hand from $(0.46, 0.37, 0.27)$ to $(0.62, -0.29, 0.39)$ m in query 1 and from $(0.54, 0.42, 0.39)$ to $(0.51, -0.23, 0.52)$ m in query 2. The direct edge swings the forearm through the wall.

Window scene (`window.yaml`): a window frame in the plane $x = 0.45$ m with an opening about 0.6 by 0.5 m, with the right post tilted 15 degrees, and a small shelf with a ball behind it. In query 1 the arm starts rotated to the left (hand near $(0.07, 0.55, 0.37)$) and reaches through the window to $(0.56, 0.08, 0.44)$. In query 2 it starts to the right (hand near $(0.20, -0.45, 0.38)$) and reaches higher to $(0.62, 0.08, 0.50)$. Both goals have only 4 to 5 cm of clearance and the direct edge hits the frame.

With goal bias 0.05 and 10 s, RTP solved the wall scene 20/20 times (seeds 1 to 10 per query, median 0.56 s) but the window scene only 14/20 (query 1 9/10, query 2 5/10, median 2.55 s for the exact runs). To get through the window the whole forearm has to line up with the opening, and since RTP connects a random vertex to a random sample with a full length edge, almost all of those edges clip the frame. Failed runs had

around 800 vertices, mostly in the open space in front of the robot, and often the approximate path was just the start, since in 7D the start is usually closer to the goal than any uniform sample. The paths look bad, as expected: every edge goes to a random configuration, so the arm makes big swings and the gripper traces large loops (Figure 1). The straight line distance for our queries is 1.2 to 1.6 rad but the RTP paths were 20 to 25 rad, and minimum clearance on the saved paths was only 0.5 to 3.7 cm.

## Benchmarking RTP, RRT, RRT-Connect and PRM (Exercise 3)

**Setup.** We added 20 settings to `configs/planners.json`: RTP with goal bias $gb \in \{0.01, 0.05, 0.1, 0.2, 0.5\}$, RRT with range $r \in \{0.2, 0.4, 1.0, 2.5\}$ at $gb = 0.05$ plus $gb \in \{0.01, 0.2\}$ at $r = 1.0$, RRT-Connect with $r \in \{0.2, 0.4, 1.0, 2.5, 5.0\}$, and PRM with max neighbors $k \in \{5, 10, 20, 40\}$. The main run used `benchmark.py` on the five clutter queries with 5 trials each (25 runs per setting) and a 5 s limit for everyone, which gives `results/benchmark.db`. We reran the same thing with 1 s and 20 s limits, and once on the shelf scene at 5 s as a harder check (`results/benchmark_t1.db`, `_t20.db`, `_shelf.db`). OMPL is seeded once with the same seed each time, so many fast runs are identical across the three time limits and only the slow runs change. The plots and tables come from these databases with our `analyze_benchmark.py`. Length, smoothness and clearance only count exact solutions. The straight line joint distance of the clutter queries is 1.75 to 2.65 rad, a lower bound on path length.

**Success rate.** RRT-Connect and PRM solved all 25 runs with every setting, at every time limit and also on the shelf. RTP was also 25/25 at 5 s for every goal bias except 0.01 (24/25). RRT was the only planner that really struggled: 16/25 to 24/25 at 5 s, and still only 18/25 for $r = 0.2$ at 20 s. Its runs are bimodal. Either it finds the goal quickly (the median over all runs is 0.10 s for $r = 0.2$) or it gets stuck and grows a tree of 5,000 to 40,000 states without ever connecting, so more time barely helps. Our guess is that the goal configurations in clutter sit close to obstacles, and with goal bias RRT always extends the nearest vertex toward the goal, which keeps hitting the same obstacle, while the rest of the time the Voronoi bias pulls the tree out toward the faces of the big 7D joint box. RTP connects the goal to a random vertex instead of the nearest, so it tries many different approach directions, and in a scene this open one of them usually works. On the shelf RRT got worse (14/25 to 23/25) while RTP with $gb \geq 0.05$, RRT-Connect and PRM stayed at 25/25.

**Planning time.** RRT-Connect was fastest by a lot, with medians of 0.019 to 0.035 s for all ranges. Growing two trees and greedily connecting them is very effective here because the free space is mostly one big connected region. PRM took 0.053 to 0.086 s and had only about 5 milestones when it found the solution. RTP's speed depends almost completely on goal bias: 1.119 s at 0.01, 0.240 s at 0.05 and 0.044 s at 0.5, with median tree sizes going from 96 to 5 vertices. A high bias means RTP often tries to jump straight to the goal from a random vertex, and in clutter that usually works after just a few vertices. At 0.01 it rarely tries the goal and has to wait for a lucky sample, which is why it only solved 10/25 with a 1 s limit. For RRT, larger goal bias at $r = 1.0$ lowered the median time (0.556 s at 0.01 vs 0.092 s at 0.2) but did not fix the stuck runs.

**Path length.** This is where RTP is clearly worst: medians of 10.9 to 24.4 rad, which is 5 to 10 times the straight line distance. Every RTP edge goes to a uniformly random configuration anywhere in the joint box, so the path wanders even when only a few edges are used. A higher goal bias gives shorter paths (24.43 rad at 0.01 down to 10.94 rad at 0.5) only because fewer random edges are needed before the jump to the goal. RRT and RRT-Connect with small ranges gave the shortest paths, 3.90 rad for RRT-Connect $r = 0.2$ and 4.24 rad for RRT $r = 0.2$, because short steps toward samples from the nearest vertex keep the tree close to the start and goal. Larger range made paths longer for both (RRT-Connect 8.37 rad at $r = 5.0$). PRM was in between at 7.75 to 8.73 rad. It stops as soon as start and goal are connected through a handful of milestones, so it never builds a dense roadmap that could give a short path.

| | exact solutions (of 25) | | | | clutter, 5 s, medians | | | |
|---|---|---|---|---|---|---|---|---|
| setting | 1 s | 5 s | 20 s | shelf 5 s | time (s) | length (rad) | smoothness | min clear. (cm) |
| RTP gb=0.01 | 10 | 24 | 25 | 21 | 1.119 | 24.43 | 0.73 | 5.4 |
| RTP gb=0.05 | 20 | 25 | 25 | 25 | 0.240 | 15.92 | 0.67 | 5.9 |
| RTP gb=0.1 | 25 | 25 | 25 | 25 | 0.199 | 17.12 | 0.66 | 8.1 |
| RTP gb=0.2 | 25 | 25 | 25 | 25 | 0.076 | 13.30 | 0.43 | 7.7 |
| RTP gb=0.5 | 25 | 25 | 25 | 25 | 0.044 | 10.94 | 0.49 | 6.9 |
| RRT gb=0.05 r=0.2 | 17 | 17 | 18 | 14 | 0.104 | 4.24 | 667.67 | 1.2 |
| RRT gb=0.05 r=0.4 | 16 | 16 | 21 | 15 | 0.557 | 4.59 | 79.96 | 1.6 |
| RRT gb=0.01 r=1.0 | 15 | 21 | 23 | 20 | 0.556 | 8.42 | 11.57 | 8.8 |
| RRT gb=0.05 r=1.0 | 17 | 24 | 25 | 18 | 0.215 | 7.36 | 10.69 | 4.1 |
| RRT gb=0.2 r=1.0 | 19 | 22 | 24 | 18 | 0.092 | 6.55 | 10.23 | 2.7 |
| RRT gb=0.05 r=2.5 | 19 | 23 | 25 | 23 | 0.336 | 9.23 | 2.03 | 9.8 |
| RRT-Connect r=0.2 | 25 | 25 | 25 | 25 | 0.035 | 3.90 | 549.15 | 0.4 |
| RRT-Connect r=0.4 | 25 | 25 | 25 | 25 | 0.029 | 4.80 | 98.61 | 0.6 |
| RRT-Connect r=1.0 | 25 | 25 | 25 | 25 | 0.019 | 4.44 | 8.87 | 3.6 |
| RRT-Connect r=2.5 | 25 | 25 | 25 | 25 | 0.022 | 5.98 | 1.49 | 4.9 |
| RRT-Connect r=5.0 | 25 | 25 | 25 | 25 | 0.028 | 8.37 | 0.53 | 9.1 |
| PRM k=5 | 25 | 25 | 25 | 25 | 0.053 | 7.75 | 0.45 | 8.7 |
| PRM k=10 | 25 | 25 | 25 | 25 | 0.065 | 7.91 | 0.44 | 9.9 |
| PRM k=20 | 25 | 25 | 25 | 25 | 0.084 | 8.72 | 0.44 | 4.6 |
| PRM k=40 | 25 | 25 | 25 | 25 | 0.086 | 8.73 | 0.40 | 3.0 |

**Table 1:** Exact solutions out of 25 runs (5 clutter queries $\times$ 5 trials) for each time limit and for the shelf scene, and median metrics for the main 5 s clutter run. Time is over all runs, so failed runs count as 5 s. The other three metrics are over exact runs only.

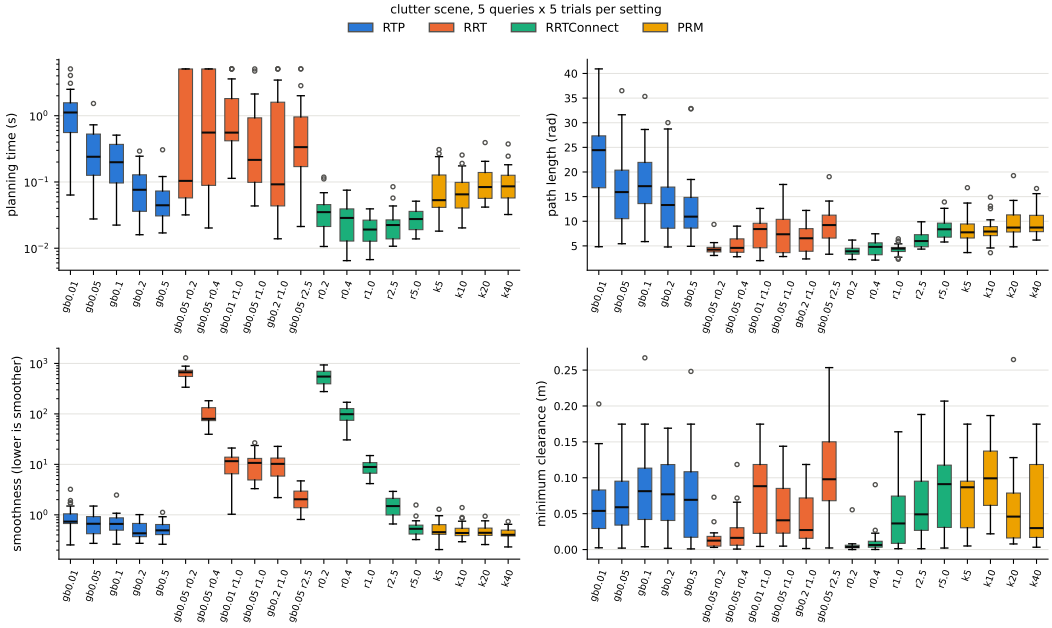
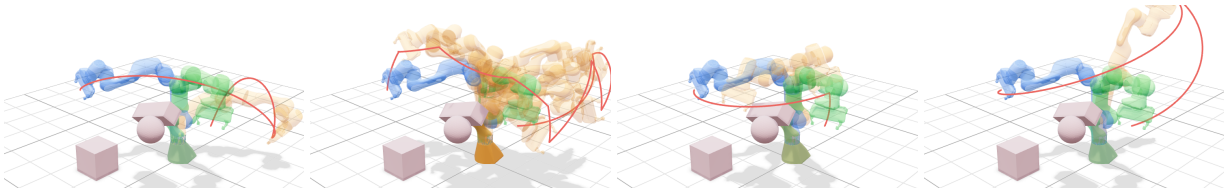



**Figure 2:** Main clutter benchmark (5 s limit, 25 runs per setting). Planning time and smoothness are on log scales. Path quality boxes use exact solutions only.



(a) RTP gb=0.2, 6.51 rad (b) RRT r=1.0, 10.63 rad (c) RRT-Connect r=1.0, 2.92 rad (d) PRM k=10, 7.13 rad

**Figure 3:** Paths for clutter query 1 with our selected settings (5 s limit, seed 1), from `plan_arm.py`. Same colors as Figure 1.

**Smoothness.** OMPL's smoothness adds up a turning term at every interior waypoint divided by the lengths of the neighboring edges, so it heavily punishes many short edges. That is why the short range RRT and RRT-Connect paths score 98 to 668 even though they look reasonable, while paths with 3 to 6 long edges score below 1. PRM $k = 40$ (0.40) and RTP $gb = 0.2$ (0.43) were the best by this metric, and RRT-Connect only gets there with $r = 5.0$ (0.53). So RTP looks "smooth" here even though its paths make huge swings, which shows the metric is mostly about the number of waypoints and does not say much on its own.

**Minimum clearance.** The clearance numbers are noisy (Figure 2 shows boxes from about 0 to 20 cm for most settings), but there is a clear trend with range. Short range RRT and RRT-Connect paths come within a few millimeters of obstacles (0.4 cm median for RRT-Connect $r = 0.2$, 1.2 cm for RRT $r = 0.2$). These trees creep up to obstacle surfaces in small steps. Long edges spend most of their length in open space, so longer ranges had more clearance (9.1 cm for RRT-Connect $r = 5.0$, 9.8 cm for RRT $r = 2.5$). RTP was in the middle at 5.4 to 8.1 cm and PRM had the single best median, 9.9 cm at $k = 10$, dropping to 3.0 cm at $k = 40$. None of these planners optimize clearance, so this is mostly a side effect of edge length.

**Best settings and trade-offs.** On clutter at 5 s the best settings we found per metric were: time, RRT-Connect $r = 1.0$ (0.019 s); length, RRT-Connect $r = 0.2$ (3.90 rad); smoothness, PRM $k = 40$ (0.40); clearance, PRM $k = 10$ (9.9 cm). Inside each planner the best for time were RTP $gb = 0.5$, RRT $gb = 0.2$ $r = 1.0$ (but only 22/25), RRT-Connect $r = 1.0$ and PRM $k = 5$. The main trade-off is range: short steps give short paths that hug obstacles and have tons of waypoints, long steps give longer paths with fewer waypoints and more clearance. RRT-Connect $r = 1.0$ was the best overall compromise for us (always solved, fastest, 4.44 rad, 3.6 cm). Figure 3 shows one path per planner in Viser. The RRT-Connect path is short and direct, the RRT path has 12 waypoints and visibly wanders, and RTP and PRM take a few big swings. The RRT-Connect path also had only 0.3 mm of clearance, which matches the trend above. For RTP itself, a high goal bias was better on every metric in clutter, because the scene is open enough that the direct jump to the goal usually works. In our window scene that jump almost always fails, so we would not expect high bias to help nearly as much there.

## Difficulty, time spent and contributions

Exercise 1 (implementing RTP): difficulty 5/10, about 6 hours. Exercise 2 (scenes and paths): 6/10, about 7 hours. Exercise 3 (benchmarks and analysis): 6/10, about 7 hours. The hardest part was making the window scene hard but still solvable, and figuring out why RRT kept failing in a scene the other planners found easy.

[CONTRIBUTIONS]